Fast Complete BN254 Scalar Multiplication

Optimal recoding and complete RCB addition for ArgoMAC

Kriterion research submission

5 September 2026

Abstract

The ArgoMAC baseline uses an incomplete point-addition law and 92 digits in base $2 - \omega$. This report gives a complete 91-digit construction for BN254 G1. An exact Eisenstein-norm proof establishes the digit bound. A second Lean proof establishes complete Renes–Costello–Batina addition. It covers ordinary addition, doubling, and inverse addition. The construction uses the minimum adaptor count in the fixed bidegree $(2, 2)$ Bosma–Lenstra family. Lean 4.24 checks these results without a new axiom or compiler-trusted evaluation. It also checks the 508-bit Lamport label order. The matching Rust code uses the same law and garbles independent rows with Rayon.

1 Contribution

Let r be the BN254 scalar modulus. Let ω be the nontrivial cube root used by the G1 endomorphism. Set $\beta = 2 - \omega$. The digit alphabet is

$$D = \{0, \pm 1, \pm\omega, \pm\omega^2\}.$$

The norm of β is seven.

The baseline uses 92 fixed positions. The new construction uses 91 fixed positions for every scalar. It keeps the same alphabet and input-label shape. It also replaces the exceptional affine row with one complete homogeneous row. The complete row increases the per-row adaptor count from nine to 13. Shared garbling passes and parallel row execution offset this extra work.

2 Four-corner GLV recoding

Use the Eisenstein norm

$$Q(a, b) = a^2 - ab + b^2.$$

The relevant BN254 constants are

$$\begin{aligned} A &= 147946756881789319000765030803803410728, \\ d &= 9931322734385697763, \\ C &= A + d, \\ r &= A^2 + Ad + d^2. \end{aligned}$$

The two GLV kernel vectors are

$$v_1 = (-A, d), \qquad v_2 = (-d, -C).$$

Arkworks divides the two negative quotient numerators toward zero. Thus, its GLV state is the floor representative z_0 used by the proof. The selector checks

$$z_0, \quad z_0 + v_1, \quad z_0 + v_2, \quad z_0 + v_1 + v_2.$$

Each shift is in the reconstruction kernel. Thus, every candidate represents the same scalar.

For quotient remainders $u, v \in [0, r)$, scaling maps these four states to the four corners of one Eisenstein cell. The two triangle lemmas show that one corner satisfies

$$3Q(a, b) \leq r.$$

This is the exact covering-radius step.

3 Exact 91-round bound

One recurrence round selects a unit digit and divides by β. For a state with $Q(a,b) \le m$, the selected digit gives

$$Q(a - u_d, b - v_d) \le m + 1 + \lfloor \sqrt{4m} \rfloor.$$

Exact division gives

$$Q(a - u_d, b - v_d) = 7Q(a', b').$$

Define the inverse safe bound

$$\begin{aligned} B(0) &= 2, \\ B(n+1) &= 7B(n) + 1 - \left\lfloor \sqrt{4(7B(n)+1)} \right\rfloor. \end{aligned}$$

Lean proves the generic induction rule

$$Q(a,b) \le B(n) \Longrightarrow \mathsf{after}(n+1, (a,b)) = (0,0).$$

It also checks

$$\begin{aligned} B(90) &= 807289082278604991119247062843696176836075088725595591044927635362684757370 7, \\ r &\le 3B(90). \end{aligned}$$

The numeric proof uses ten-round kernel-checked checkpoints. It does not use `native_decide`.
The main Lean results are:

- `shiftedInitialCorrect`;
- `existsCandidateNormBound`;
- `fourCandidatesSignedI128`;
- `existsCorrectShiftedInitialTerminates91`;
- `shortInitialTerminates91`;
- `shortInitialCorrect`.

The axiom audit reports only `propext`, `Classical.choice`, and `Quot.sound`.

4 Optimality

Lean checks

$$7^{90} < r < 7^{91}.$$

A fixed 90-position string over seven symbols has at most 7^{90} values. It cannot represent all r scalars. Thus, 91 is optimal for this fixed alphabet.

5 Complete homogeneous addition

Let the hidden offset be $K = (A : B : C)$. Let (u, v) be the selected endomorphism image of the evaluator input. Use homogeneous coordinates for

$$Y^2 Z = X^3 + 3Z^3.$$

An affine point is $(x : y : 1)$. The group identity is $(0 : 1 : 0)$.
RCB Algorithm 7 gives these three polynomials:

$$\begin{aligned} X &= -9AB - 18BCu - 18ACv + (B^2 - 9C^2)uv + ABv^2, \\ Y &= -81C^2 + 27A^2u + 27ACu^2 + B^2v^2, \\ Z &= 9BC + (B^2 + 9C^2)v + BCv^2 + 3A^2uv + 3ABu^2. \end{aligned}$$

One fresh nonzero value ρ multiplies all three coordinates. This common factor preserves the represented projective point. The zero digit returns $\rho(A, B, C)$.

The decoder keeps Z until the final projection. For $Z \neq 0$, it validates $(X/Z, Y/Z)$ and returns that point. For $Z = 0$, it accepts only $X = 0$ and $Y \neq 0$. This case returns the group identity. It rejects all other zero-denominator triples.

Lean proves the homogeneous curve equation. It proves that the output is nonzero and projectively nonsingular. It then proves that projective decode equals Mathlib group addition for every valid affine pair. The proof has separate cases for different x-coordinates, doubling, and inverse inputs. The main results are:

- `outputNonsingular`;
- `outputRepresentsSum`;
- `decodeHomogeneous_eq_toAffine`;
- `decodeEvaluateRowSome`;
- `decodeEvaluateRowNone`;
- `perfectCorrectness`.

The axiom audit reports only `propext`, `Classical.choice`, and `Quot.sound`.

6 Adaptor optimum

The complete Bosma–Lenstra family has projective parameters $[\alpha : \beta : \gamma]$. Completeness requires $\beta \neq 0$. Normalize $\beta = 1$. Under the exact `Biquadratic` support rule, the adaptor counts are

$$D(0, 0) = 13, \qquad D(0, \gamma) = 14 \text{ for } \gamma \neq 0, \qquad D(\alpha, \gamma) = 15 \text{ for } \alpha \neq 0.$$

Thus, RCB parameters $[0 : 1 : 0]$ give the unique minimum of 13. This result covers fixed bidegree $(2, 2)$ laws in the current masking architecture.

A second lower bound covers private role-preserving linear rows. A row with t scalar adaptors exposes $14 + t$ selected field values. Its adaptor randomness has dimension at most $2t - 2$. Only the three output coordinates can stay unmasked. Thus, $2t - 2 \geq 11 + t$, so $t \geq 13$. A correct ten-adaptor sharing attempt violates this bound and exposes four extra coefficient invariants. The current row reaches 13. This result does not exclude a nonlinear garbling architecture.

7 Circuit cost

Item	Original 92-row call	Complete 91-row call	Change
Output MAC rows	92	91	-1
Digit adaptors	833	1,188	$+355$
Bit adaptors	211,582	301,752	$+90,170$
Fixed-permutation applications	6,322,060	1,508,760	$-4,813,300$
Public permutation instances	12,495	17,820	$+5,325$

The first column counts repeated calls in the unoptimized baseline. The last column uses one shared pass for each bit and point row. The public instance count increases because the complete law needs more live support.

The Lean direct-table benchmark used 21 measured runs on Apple M5. The complete construction has a 231 ms median. The original baseline has a 989 ms median. The complete construction is 4.28 times faster. It reduces the median by 76.6 percent.

8 Rust row parallelism

The Rust implementation allocates one job for each output row. It allocates the three child nonce domains before Rayon starts. It samples one fresh 32-byte ChaCha12 root from the caller RNG. Row j uses stream

$$\texttt{0x666d3265635f6d61} + j$$

from word position zero. Indexed collection keeps the row order. Debug builds use the same RNG and nonce schedule in serial order. Release builds use Rayon.

The root seed is erased after row construction. Jobs borrow output keys. They do not copy those secret keys into a second vector. Fixed Boolean arrays replace heap-based coefficient index sets. Tests compare one-worker and four-worker tables. A second test compares all 273 allocated child nonces with the serial schedule.

On the same Apple M5, the ARMv8 AES benchmarks used 100 Criterion samples. The incomplete 92-row serial baseline estimate was 15.934 ms. The complete 91-row, 10-worker row mean is 3.760 ms. Its 95 percent interval is 3.732 to 3.793 ms. The complete row stage is 4.24 times faster than the serial baseline.

The protocol `encode` mean is 69.766 ms. Its 95 percent interval is 69.676 to 69.872 ms. The comparable baseline is 81.309 ms. The complete implementation reduces this time by 14.2 percent. Its HMAC table has 9,668,659 bytes. This is 2,970,240 bytes larger than the incomplete 91-row table.

9 Security scope

The following invariants stay fixed:

- Every candidate reconstructs the same scalar modulo r.
- Every public circuit has exactly 91 output positions.
- The digit alphabet does not change.
- The input keeps 508 Lamport label positions.
- The fixed-key window loads stay 91, 91, and 72.
- Every support flag is public and independent of the scalar.
- One nonzero factor scales all three homogeneous coordinates.
- Each row uses a separate nonce subtree and RNG stream.
- Worker order cannot change the table or caller RNG state.

The Rust selector uses four fixed 91-round tests and `subtle::Choice`. This gives a fixed iteration count. Lean proves that every tested coordinate fits a signed 128-bit integer. The report does not claim compiler-level constant time.

The Rust checked evaluator rejects a coordinate that is at least the base-field modulus before reduction. It then checks the curve equation. A regression test uses $x = p + 1$ and $y = 2$. Reduction would otherwise map this input to the valid generator $(1, 2)$.

Lean proves that the encoding key contains the 508 Lamport label pairs in x-then-y order. It proves that encoding selects the label for each bit of `affineLamportBits`. The theorem is `Lamport.compatible`.

Lean proves exact joint-tape laws for fresh permutation and random-oracle programming. Each map sends (O, t) to the programmed oracle and the old value $O(x)$. Each map is an involution on a finite sample space. Thus, each map preserves the uniform joint PMF exactly. Programming a fixed output alone does not preserve uniformity. The permutation law also composes over every fixed finite target-tape schedule. Both oracle laws preserve the uniform fiber of fresh oracles that match a fixed query transcript. Lean also lifts an exact handler equivalence through every bounded adversary oracle program. The permutation and hash involutions extend to the complete shared simulator state. Fresh state swaps preserve every shared transcript and label invariant. Each safe ideal-oracle query commutes with the matching full-state swap. Thus, every path-safe bounded adversary program commutes with either swap in exact PMF equality. Lean proves the exact mass of every finite forbidden block set. A set with at most k values has mass at most $k \cdot 2^{-128}$. The two-point collision event therefore has mass at most 2^{-127}. Lean also sums these local masses over every finite declared collision schedule.

The complete row repairs the known perfect-correctness failure. The Rust permutation schedule still lacks a complete adaptive-privacy reduction. The worst selected branch uses the three hash slots. Its zero-query numerator is

$$3 \cdot 10692 \cdot 91^2 = 265621356 < 2^{28}.$$

The 100-bit margin is 2,814,100 numerator units. A direct five-slot union bound exceeds 2^{28}. Lean proves the 100-bit arithmetic for each exclusive three-hash or two-pad branch. The semantic reduction must still condition on the selected branch. The current Lean challenge library also defines no `Kriterion.Benchmark.garble`. The PR does not forge either missing item.

10 Reproduction

The Lean proof uses Lean 4.24 and the pinned Mathlib checkout. Run:

```
lake build Proof Correctness
lake env lean entry/tests/Correctness.lean
lake env lean AxiomCheck.lean
lake exe bench
```

The Rust validation uses Rust 1.94.0. Run:

```
cargo fmt --all -- --check
cargo clippy --all-targets --all-features -- -D warnings
cargo test --all-features
cargo test --release --all-features
RAYON_NUM_THREADS=10 cargo bench --all-features \
  --bench field_mac_to_ec_mac -- 'field_mac_to_ec_mac/g1/garble'
RAYON_NUM_THREADS=10 cargo bench --all-features \
  --bench protocol_bench -- 'encode'
```

References

J. Bosma and H. W. Lenstra, "Complete systems of two addition laws for elliptic curves," *Journal of Number Theory*, 1995.

J. Renes, C. Costello, and L. Batina, "Complete addition formulas for prime order elliptic curves," EUROCRYPT 2016.

RustCrypto, *AES 0.8.4 documentation*, ARMv8 run-time detection section.

Arkworks, *ark-ec 0.5.0*, GLV scalar decomposition implementation.